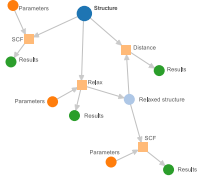
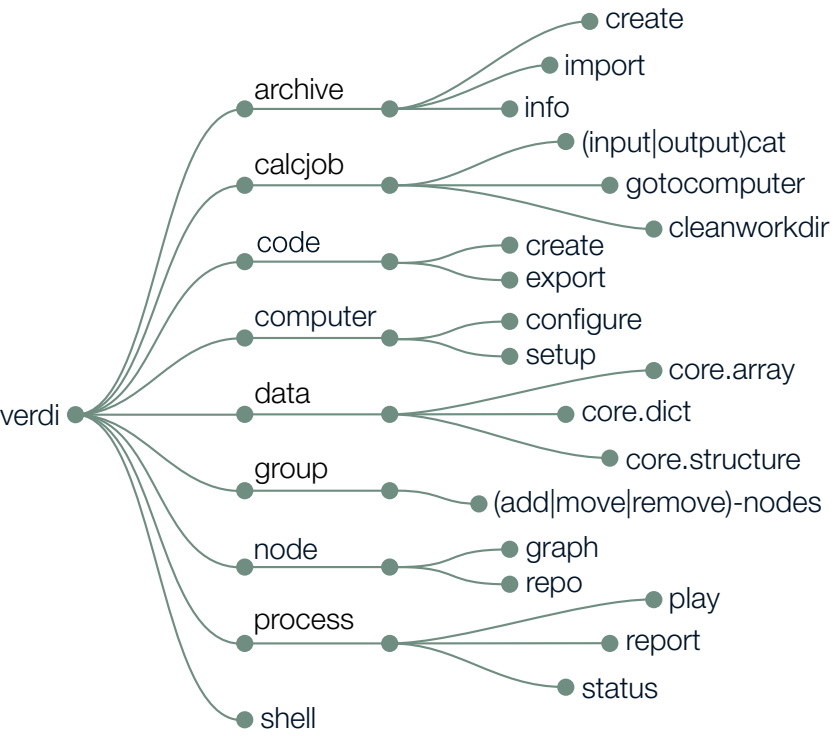
The AiiDA cheat sheet

v2.5.1



The `verdi` command-line API*



*Not exhaustive
*Most options also implement `show/list/delete`

Tools of the trade

Other `verdi` tips and tricks

Know what's there:

```
$ verdi profile list
$ verdi user list
$ verdi storage list
$ verdi plugin list aiida.calculations
$ verdi plugin list aiida.workflows
```

AiiDA to classical file tree:

```
$ verdi process dump <pk>
$ verdi calcjob dump <pk>
```

Config options, e.g. caching:

```
$ verdi config list
$ verdi config set \
    caching.default_enabled true
$ verdi config set caching.enabled_for \
    aiida.calculations:quantumespresso.pw
```

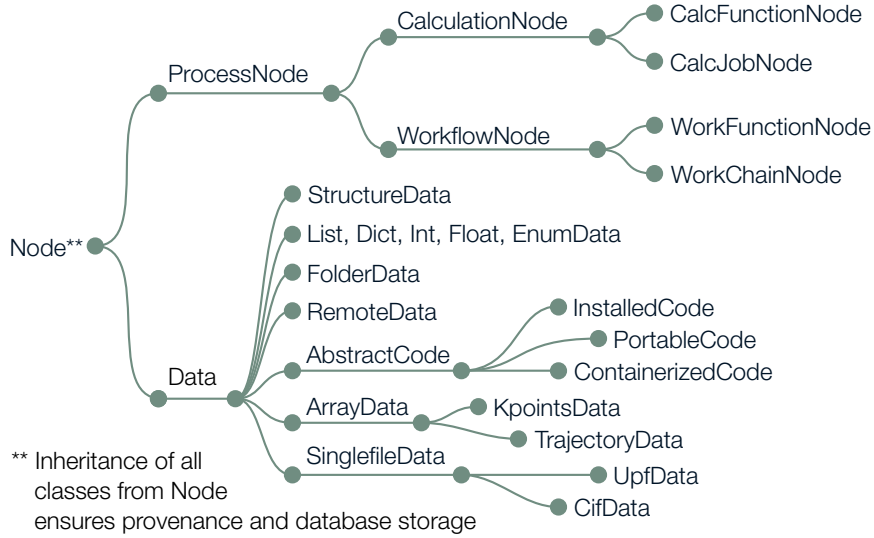
Fix what went astray:

```
$ verdi daemon restart --reset
$ verdi devel rabbitmq tasks analyze --fix
```

Share your data:

```
$ verdi archive create <archive.aiida> \
    --groups/--nodes <groups/nodes>
$ verdi archive import <archive.aiida>
```

The AiiDA Node subclasses



** Inheritance of all classes from Node ensures provenance and database storage

Additional web resources (click me)

`aiidalab` `aiida-project` `aiida-shell` `aiida-resource-registry`
`aiida-tutorials` `aiida-submission-controller` `aiida-plugin-cutter`

AiiDA Python imports

ORM, nodes, and Factories

Import aiida-core Node classes from aiida.orm:

```
from aiida.orm import Dict, CalcJobNode
```

Load Nodes via pk, UUID, or label:

```
from aiida.orm import load_node
my_node = load_node(<identifier>)
```

Import Data classes via the DataFactory:
(Note: Prefix AiiDA core types with `core`)

```
my_kpts = DataFactory("core.array.kpoints")
```

Import CalcJob classes via the CalculationFactory:

```
my_calcjob = CalculationFactory(
    "quantumespresso.pw"
)
```

Import WorkChain classes via the WorkflowFactory.

```
my_workflow = WorkflowFactory(
    "quantumespresso.pw.bands"
)
```

MARVEL
NATIONAL CENTRE OF COMPETENCE IN RESEARCH

MAX DRIVING THE EXASCALE TRANSITION